PROJECT REPORT

Simple URL Shortener

Developed Using Python Flask and SQLite

Submitted By

Hasnain Ali

Class/Section: Sp25-BCS-083

Course: Web / Backend Development Project

1. Executive Summary

The Simple URL Shortener is a lightweight web-based application developed to convert long web addresses into short, easy-to-share URLs. The project uses Python with the Flask framework for the backend, SQLite for persistent data storage, and HTML with JavaScript for the basic user interface. When a user submits a long URL, the application generates a unique six-character code, stores the relationship between the code and the original URL in a SQLite database, and returns a short URL. When the short URL is opened, Flask retrieves the corresponding original URL and redirects the user to it.

2. Project Objective

The main objective of this project is to design and implement a simple, functional URL shortening service. The specific objectives are:

- Develop a backend server using Python Flask.
- Accept long URLs through an HTTP API endpoint.
- Generate a unique short code for each submitted URL.
- Store the short-code and original-URL mapping in a SQLite database.
- Create a redirect route that sends users from a short URL to the original URL.
- Provide a simple web interface so that the system can be tested without a separate API client.
- Demonstrate basic concepts of web routing, REST-style requests, database operations, and URL redirection.

3. Project Requirements

3.1 Functional Requirements

- The system shall accept a valid long URL from the user.
- The system shall generate a unique short code.
- The system shall store the URL mapping in a database.
- The system shall return a short URL to the user.
- The system shall redirect a valid short URL to its original URL.
- The system shall display an error message when a required URL is not provided.
- The system shall return a not-found response when an unknown short code is requested.

3.2 Non-Functional Requirements

- The application should be simple and easy to use.
- The source code should remain understandable for a beginner-level web development project.
- The database should persist URL mappings between application runs.
- The application should provide clear responses for successful and unsuccessful requests.

4. Tools and Technologies Used

Tool / Technology	Purpose	Role in Project
Python	Programming language	Backend application logic
Flask	Python web framework	Routes, API endpoint, and server
SQLite	Relational database	Stores URL mappings

HTML	Markup language	Frontend structure
JavaScript	Client-side scripting	Sends request and displays result
VS Code	Code editor	Development and project management
Python venv	Virtual environment	Isolates project dependencies
Web Browser	Application testing	Tests frontend and redirect behavior

5. System Architecture

The application follows a simple client-server architecture. The browser acts as the client. Flask receives HTTP requests and executes the required backend logic. SQLite stores the mapping between each generated short code and its original URL.

- Client Layer: HTML and JavaScript interface displayed in the browser.
- Application Layer: Flask routes and Python functions process requests.
- Data Layer: SQLite database stores short codes and original URLs.

Basic data flow:

User → Web Interface → Flask API → SQLite Database → Short URL → Redirect to Original URL

6. Project Structure

The final project directory contains the following main files and folders:

- app.py — main Flask application containing routes, database initialization, URL generation, and redirect logic.
- templates/index.html — basic frontend page containing the URL input field, button, and JavaScript request logic.
- urls.db — SQLite database automatically created by the application to store URL mappings.
- requirements.txt — lists the Flask dependency required by the project.
- venv/ — Python virtual environment containing isolated project packages.

7. Implementation Details

7.1 Flask Application

The Flask application is initialized in app.py. Flask provides the routing system and handles HTTP requests. The application uses separate routes for the home page, URL-shortening operation, and redirection.

7.2 Database Initialization

A SQLite database named urls.db is initialized when the application starts. The urls table contains an automatically generated numeric ID, a unique short_code, and the original_url.

Field	Type	Description
id	INTEGER	Primary key and automatically incremented identifier
short_code	TEXT	Unique six-character code
original_url	TEXT	Original long URL supplied by

		the user

7.3 Short Code Generation

The application generates a six-character code using uppercase letters, lowercase letters, and digits. The generated code is used as the unique identifier for the original URL.

7.4 API Endpoint

The POST /shorten endpoint accepts JSON data containing a URL. The server validates that a URL value is present, generates a short code, stores the mapping in SQLite, and returns the original URL, short code, and complete short URL in JSON format.

7.5 Redirect Route

The /<short_code> route searches the SQLite database for the requested short code. If a matching record is found, Flask redirects the browser to the stored original URL. If no record exists, the application returns a 404 response indicating that the short URL was not found.

7.6 Frontend

The frontend provides a simple input field and a Shorten URL button. JavaScript sends a POST request to the /shorten endpoint using JSON. The returned short URL is then displayed as a clickable link.

8. Complete Working Procedure

1. The user opens the URL Shortener homepage in a web browser.
2. The user enters a long URL in the input field.
3. The user clicks the Shorten URL button.
4. JavaScript sends the URL to the Flask /shorten endpoint.
5. Flask validates the submitted data.
6. A unique six-character short code is generated.
7. The short code and original URL are stored in the SQLite database.
8. Flask returns the generated short URL.
9. The frontend displays the short URL as a clickable link.
10. When the short URL is opened, Flask searches the database for its code.
11. Flask redirects the browser to the corresponding original URL.

9. API Documentation

Method	Endpoint	Purpose	Expected Result
GET	/	Open the frontend	URL Shortener page
POST	/shorten	Create a short URL	JSON response containing short URL
GET	/<short_code>	Open a short URL	Redirect to original URL

10. Example API Request

Example JSON input:

```
{ "url": "https://www.youtube.com/" }
```

The response contains the original URL, generated short code, and short URL.

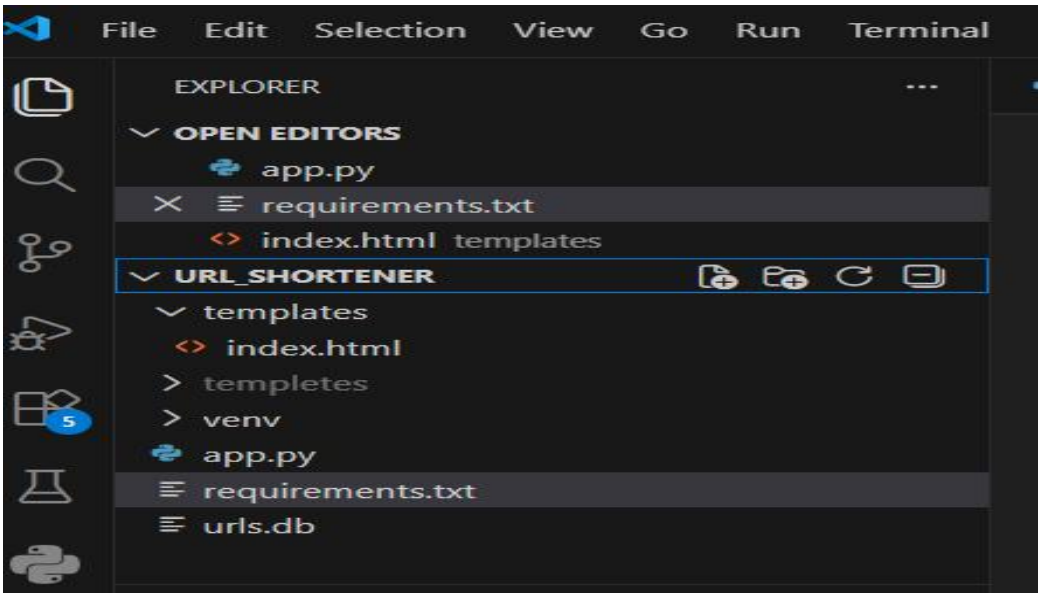
11. Testing and Verification

Test Case	Input / Action	Expected Result	Status
Open application	Visit http://127.0.0.1:5000	Homepage is displayed	Passed
Submit URL	Enter a valid long URL	Short URL is generated	Passed
Database storage	Generate a short URL	Mapping is saved in SQLite	Passed
Redirect	Open generated short URL	Original website opens	Passed
Invalid short code	Open unknown code	404 / not found response	Implemented

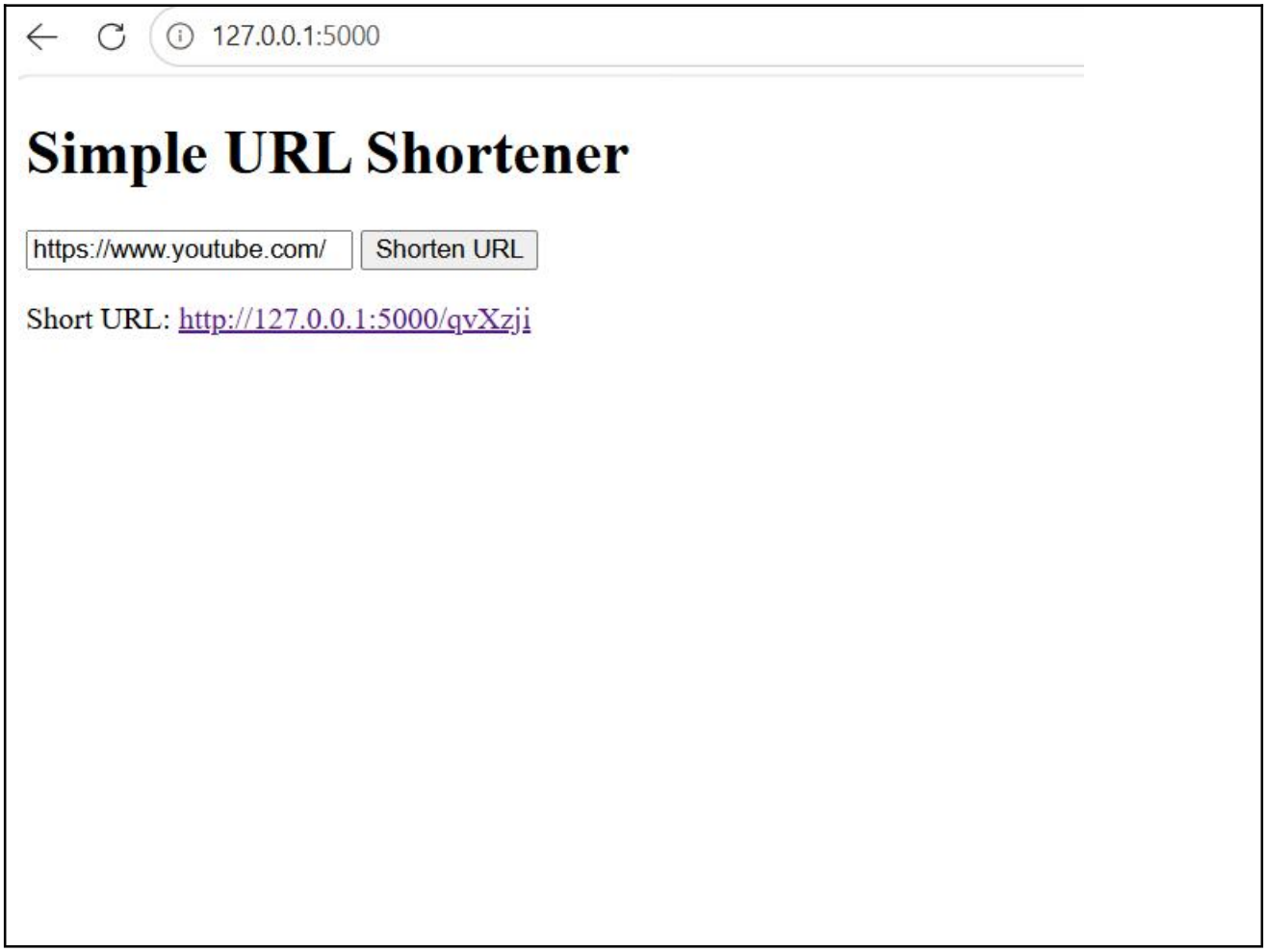
12. Screenshots / Evidence

The following screenshots should be inserted in the spaces provided below. These screenshots document the development environment, working interface, and successful URL redirection.

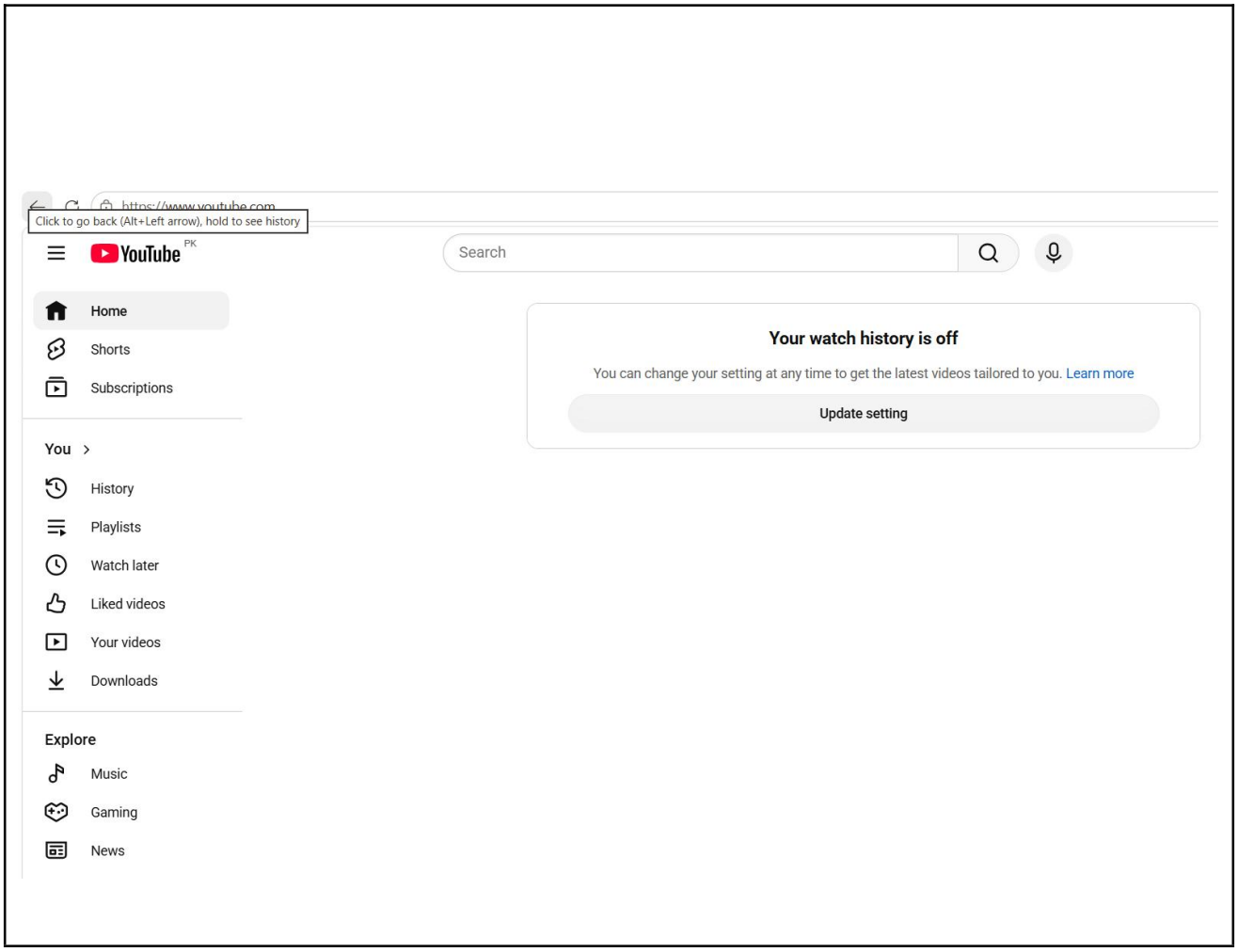
Screenshot 1: Project Structure in Visual Studio Code



Screenshot 2: URL Shortener Interface and Generated Short URL



Screenshot 3: Successful Redirection to the Original Website



13. Limitations and Possible Improvements

- The current application is intended for local academic demonstration and does not include production-grade authentication.
- URL validation can be strengthened with stricter URL parsing and allowed schemes.
- A production deployment would require stronger collision handling and database constraints.
- A public version could add analytics such as click counts and creation dates.
- A custom domain and HTTPS could be configured for real-world deployment.
- The frontend can be improved with CSS for a more polished user interface.

14. Conclusion

The Simple URL Shortener project was successfully designed and implemented using Python Flask, SQLite, HTML, and JavaScript. The system meets the core requirements of accepting a long URL, generating a unique short code, storing the URL mapping in a database, producing a short URL, and redirecting the short URL to the original destination. The project demonstrates practical understanding of Flask routing, HTTP requests, JSON data, database operations, frontend interaction, and URL redirection.

15. References

- Python Documentation — Python programming language reference and standard library.
- Flask Documentation — Web application framework for Python.
- SQLite Documentation — Lightweight relational database engine.
- MDN Web Docs — HTML, JavaScript, Fetch API, and HTTP concepts.